



Lecture 2

Activity Diagram

Barbora Bühnová, Stanislav Chren

Recap - Textual use case specification



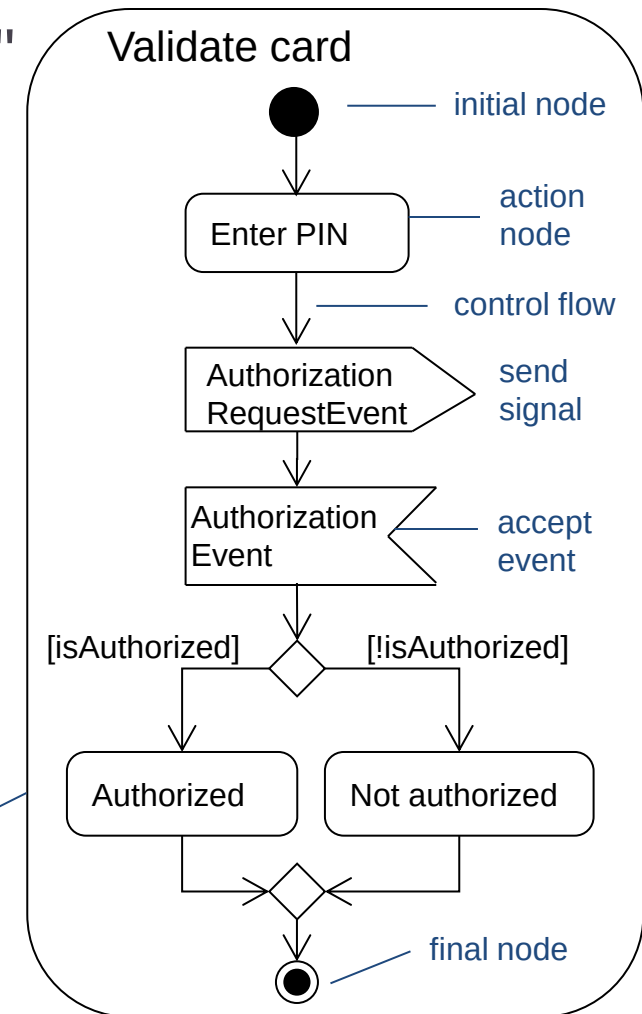
Use Case ID	UC9
Brief Description	Support Managers updates the information or status of the issue ticket to reflect the work progress on the issue
Primary Actors	Support Technician
Secondary Actors	Issue Reporter
Preconditions	Issue Reporter is logged in the system
Main Flow of Events	1. Use case starts when Support Technician selects "Update Tickets" option in the menu 2. <i>INCLUDE</i>(Find Tickets) 3. <i>IF</i> at least one ticket assigned to the Issue Technician is found 3.1. Support Technician selects one of the tickets from the displayed list 3.2. System shows detailed information about the issue ticket 3.3. Support Technician selects the "Edit Status" option. 3.4. System displays an editable form for the ticket details. 3.5. Support Technician updates the ticket information or its status. <i>EXTENSION POINT</i>(TicketIsClosed) 3.6. System stores the changes in the database
Alternative Flows	Issue Reporter can log out at any time
Post-conditions	The status of the issue ticket was updated.

What are activity diagrams?



- ✧ Activity diagrams are "OO flowcharts"
- ✧ They allow us to model a process as a collection of nodes and edges between those nodes
- ✧ Use activity diagrams to model the behavior of:
 - use cases
 - collaborations (of classes, components, people, departments)
 - operations and methods
 - business processes

activity



Activities

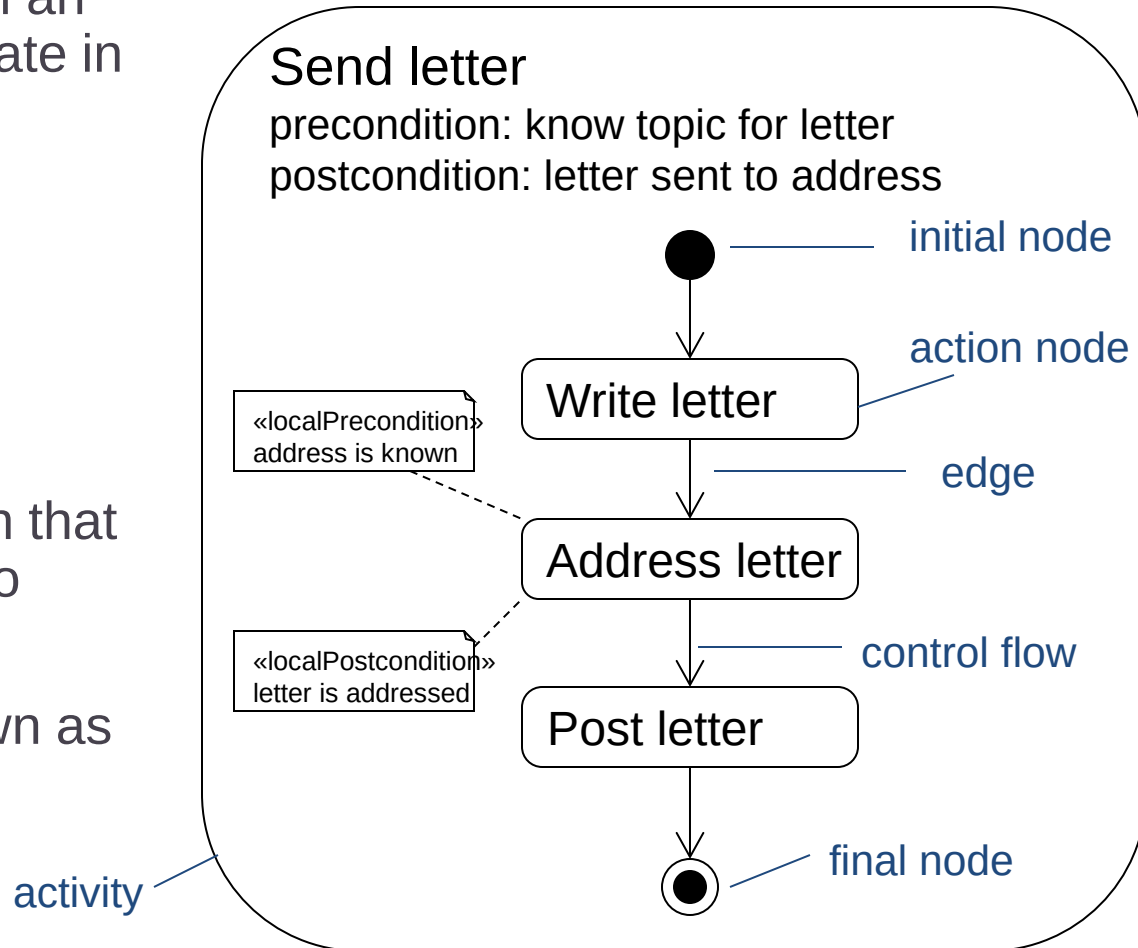


- ✧ Activities are networks of nodes connected by edges
- ✧ There are three categories of node:
 - **Action nodes** – represent atomic units of work within the activity
 - **Control nodes** - control the flow through the activity
 - **Object nodes** - represent the flow of objects around the activity
- ✧ Edges represent flow through the activity
- ✧ There are two categories of edge:
 - **Control flows** - represent the flow of control through the activity
 - **Object flows** - represent the flow of objects through the activity
- ✧ What is the difference between an action and activity?
How can I recognize one from another in the diagram?

Activity diagram syntax



- ✧ Activities usually start in an **initial node** and terminate in a **final node**
- ✧ Activities can have **preconditions** and **postconditions**
- ✧ When an action node finishes, it emits a token that may traverse an edge to trigger the next action
- ✧ This is sometimes known as a **transition**



Activity diagram semantics



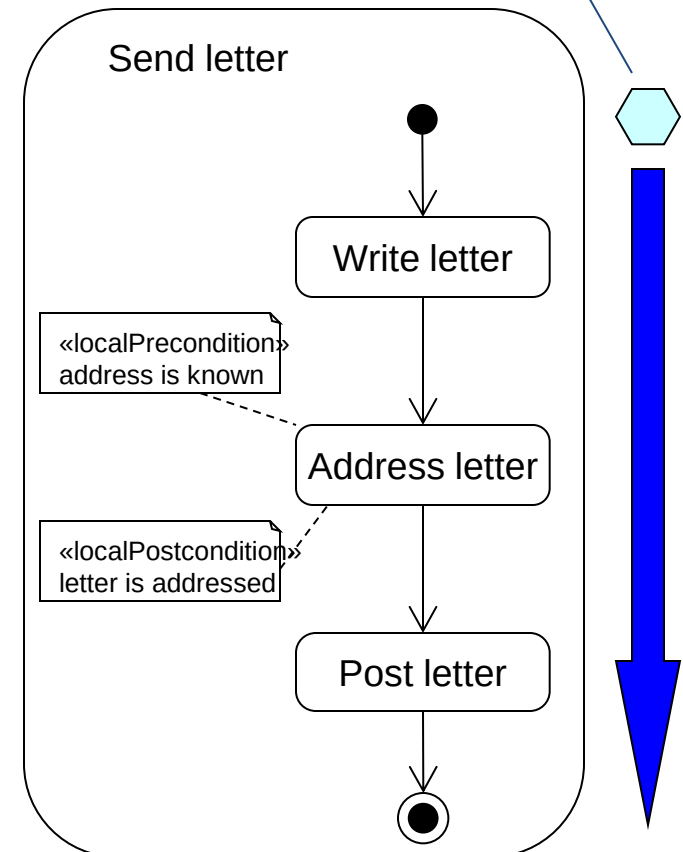
✧ The **token game**

- Token – an object, some data or a focus of control
- Imagine tokens flowing around the activity diagram

✧ Tokens traverse from a source node to a target node via an edge

- The source node, edge and target node may all have constraints controlling the movement of tokens
- All constraints **must** be satisfied before the token can make the traversal

imaginary flow of control token



Action nodes

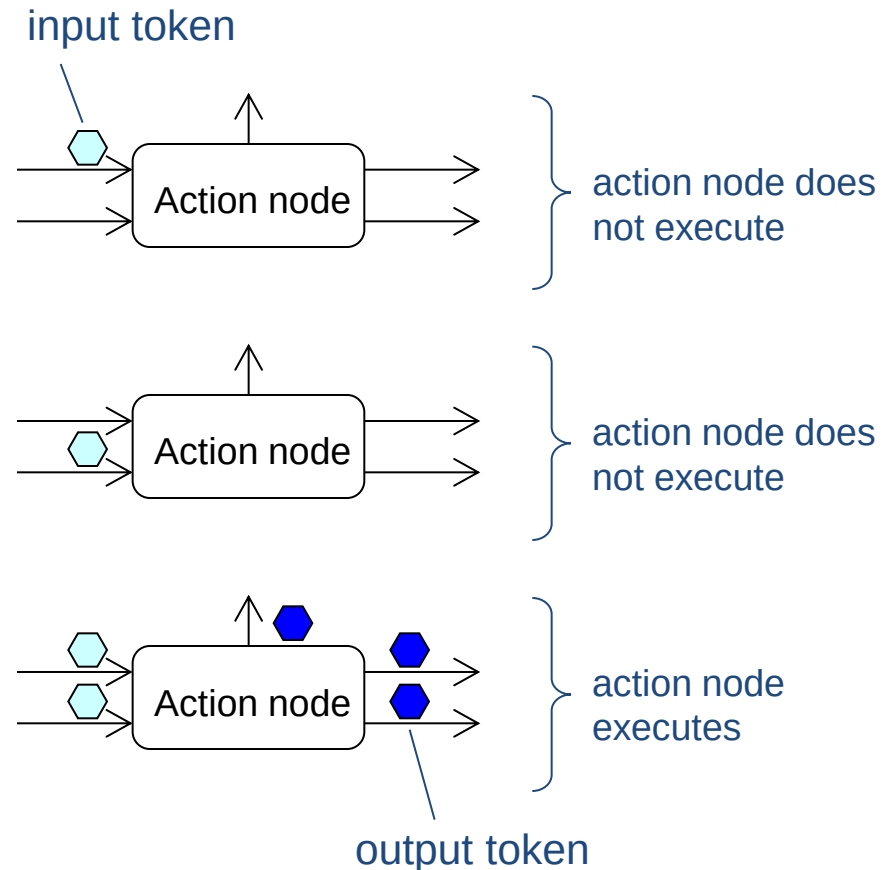


✧ Action nodes offer a token on **all** of their output edges when:

- There is a token **simultaneously** on each input edge
- The input tokens satisfy all preconditions specified by the node

✧ Action nodes:

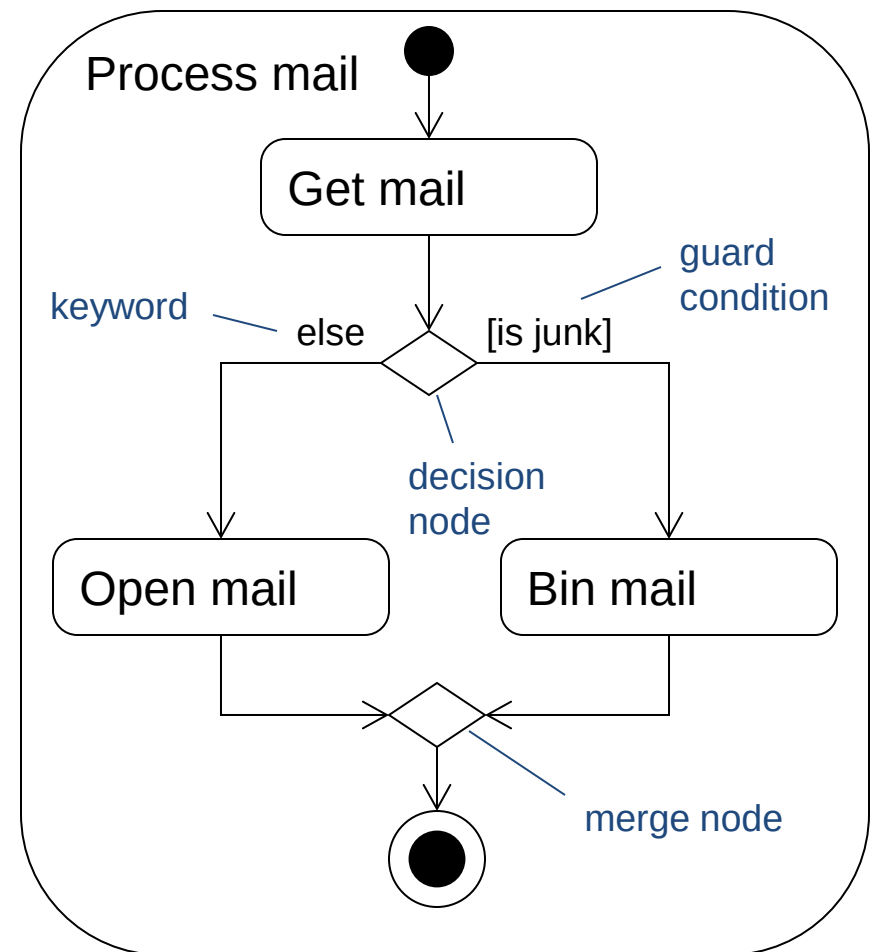
- Perform an implicit fork on their output edges when they have finished executing



Decision and merge nodes



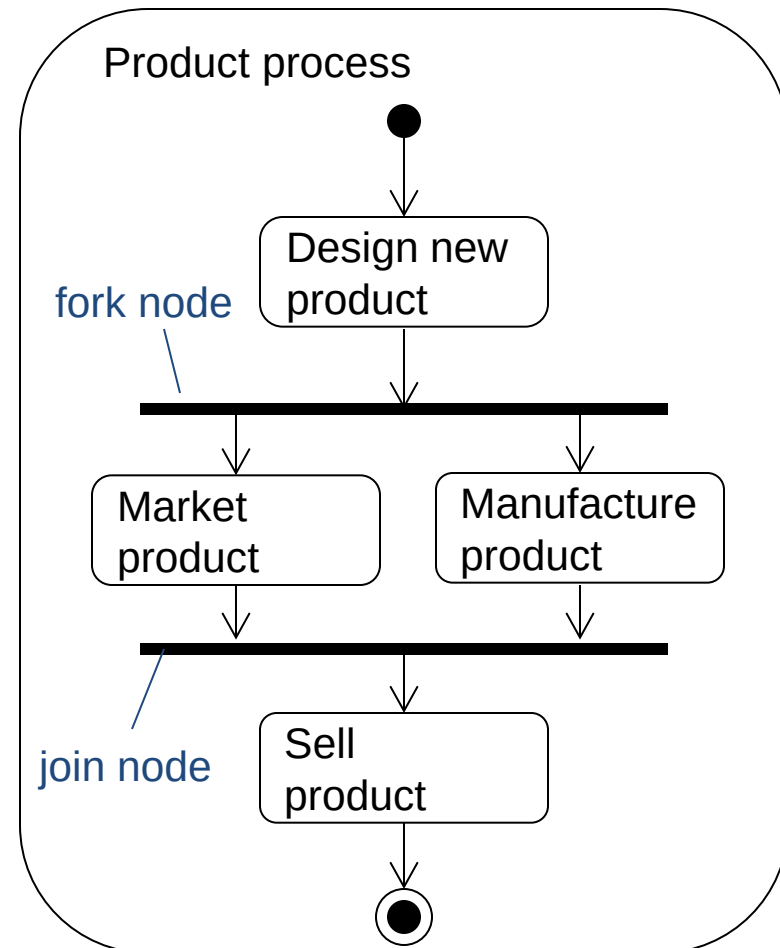
- ✧ A **decision node** is a control node that has one input edge and two or more alternate output edges
 - Each edge out of the decision is protected by a **guard condition**
 - guard conditions must be mutually exclusive
 - The edge can be taken if and only if the guard condition evaluates to true
 - The keyword **else** specifies the path that is taken if *none* of the guard conditions are true
- ✧ A **merge node** allows through any of several alternate flows
 - It has two or more input edges and exactly one output edge



Fork and join nodes – concurrency



- ✧ **Forks nodes** model concurrent flows of work
 - Tokens on the single input edge are replicated at the multiple output edges
- ✧ **Join nodes** synchronize two or more concurrent flows
 - Joins have two or more incoming edges and exactly one outgoing edge
 - A token is offered on the outgoing edge when there are tokens on all the incoming edges i.e. when the concurrent flows of work have all finished



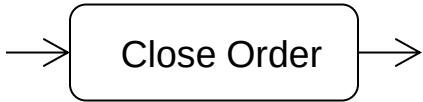
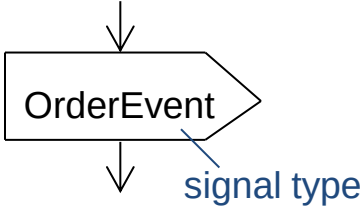
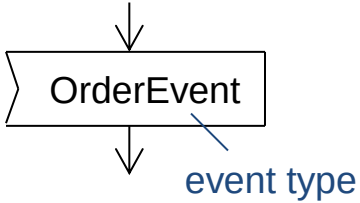
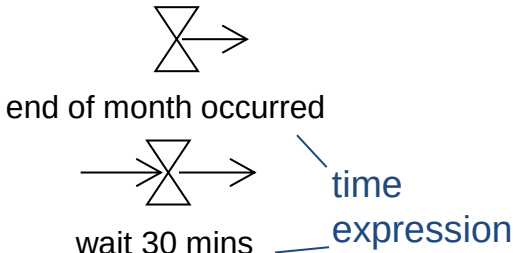
Control nodes



control node syntax	control node semantics	
	Initial node – indicates where the flow starts when an activity is invoked	
	Activity final node – terminates an activity	Final nodes
	Flow final node – terminates a specific flow within an activity. The other flows are unaffected	
	Decision node– guard conditions on the output edges select one of them for traversal May optionally have inputs defined by a «decisionInput»	
	Merge node – allows through any of its input edges	
	Fork node – splits the flow into multiple concurrent flows	
{join spec} 	Join node – synchronizes multiple concurrent flows May optionally have a join specification to modify its semantics	

Types of action node



action node syntax	action node semantics
	Call action - invokes an activity, a behavior or an operation. The most common type of action node. See next slide for details.
	Send signal action - sends a signal asynchronously. The sender <i>does not</i> wait for confirmation of signal receipt. It may accept input parameters to create the signal
	Accept event action - waits for events detected by its owning object and offers the event on its output edge. Is enabled when it gets a token on its input edge. If there is <i>no</i> input edge it starts when its containing activity starts and is <i>always</i> enabled.
 end of month occurred wait 30 mins	Accept time event action - waits for a set amount of time. Generates time events according to it's time expression.

Call action node syntax



✧ The most common type of node

✧ Call action nodes may invoke:

- an activity
- a behavior
- an operation

✧ They may contain code fragments in a specific programming language

- The keyword 'self' refers to the context of the activity that owns the action

Raise Order 

call an activity
(note the rake icon)

Close Order

call a behavior

getBalance():double
(Account::)

operation name
class name
(optional)

Get Balance
(Account::getBalance():double)

node name
operation name
(optional)

```
if self.balance <= 0:  
    self.status = INCREDIT  
else  
    self.status = OVERDRAWN
```

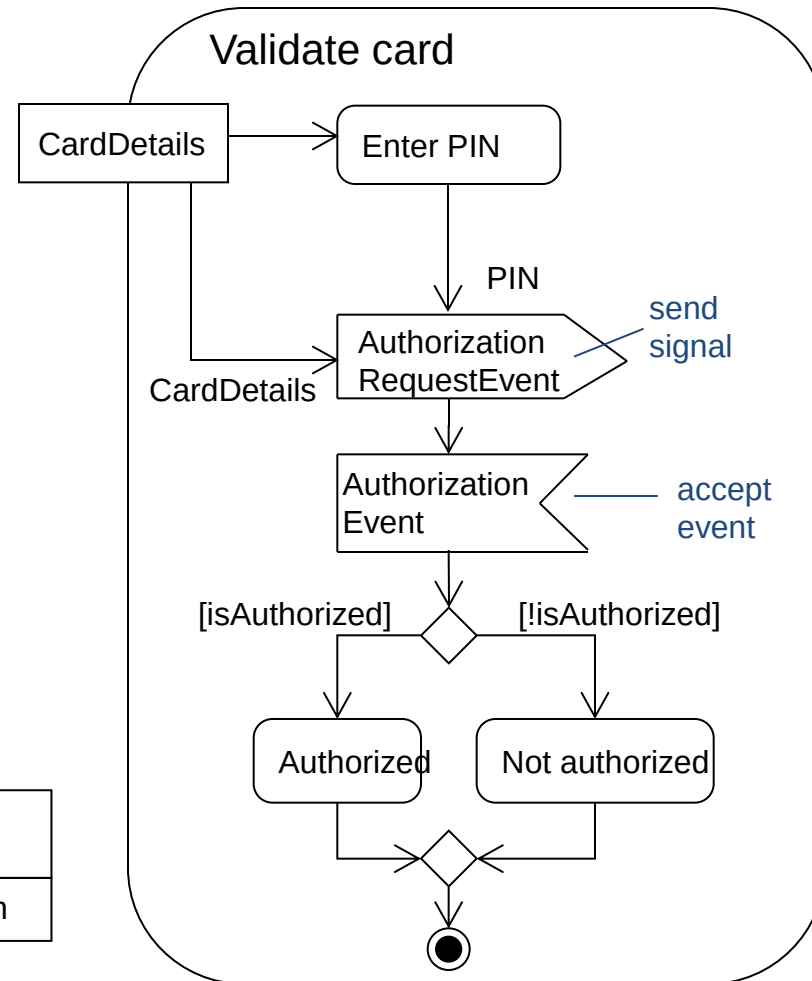
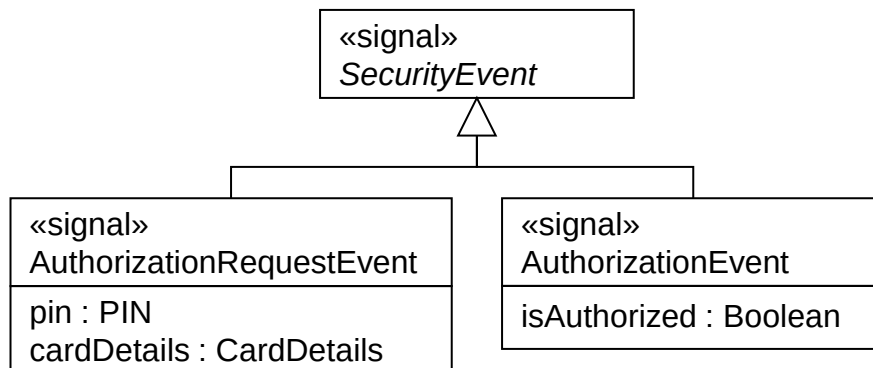
programming
language
(e.g. Python)

call an
operation

Sending signals and accepting events



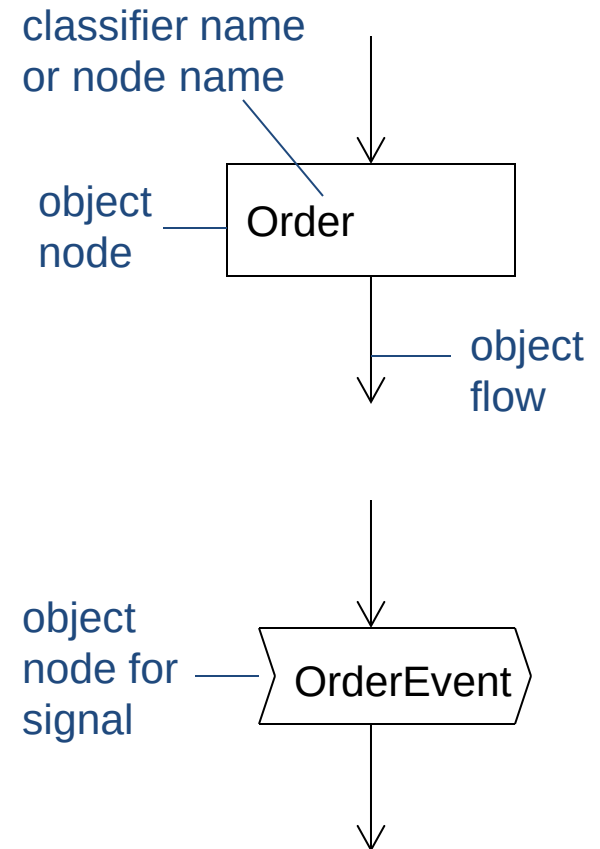
- ✧ Signals represent information passed asynchronously between objects
 - This information is modelled as attributes of a signal
 - A signal is a classifier stereotyped «signal»
- ✧ The accept event action asynchronously accepts event triggers which may be signals or other objects



Object nodes



- ✧ Object nodes indicate that instances of a particular classifier may be available
 - If no classifier is specified, then the object node can hold any type of instance
- ✧ Multiple tokens can reside in an object node **at the same time**
 - The upper bound defines the maximum number of tokens (infinity is the default)
- ✧ Tokens are presented to the single output edge according to an ordering:
 - FIFO – first in, first out (the default)
 - LIFO – last in, first out
 - Modeler defined – a selection criterion is specified for the object node



Object node syntax



✧ Object nodes have a flexible syntax. You may show:

- upper bounds
- ordering
- sets of objects
- selection criteria
- object in state

Order

order objects may be available

Order

{upperBound = 12}

zero to 12 Order objects may be available

Order

{ordering = LIFO}

last Order object in is the first out (FIFO is the default)

Set of Order

sets of Order objects may be available

«selection»
monthRaised = "Dec"

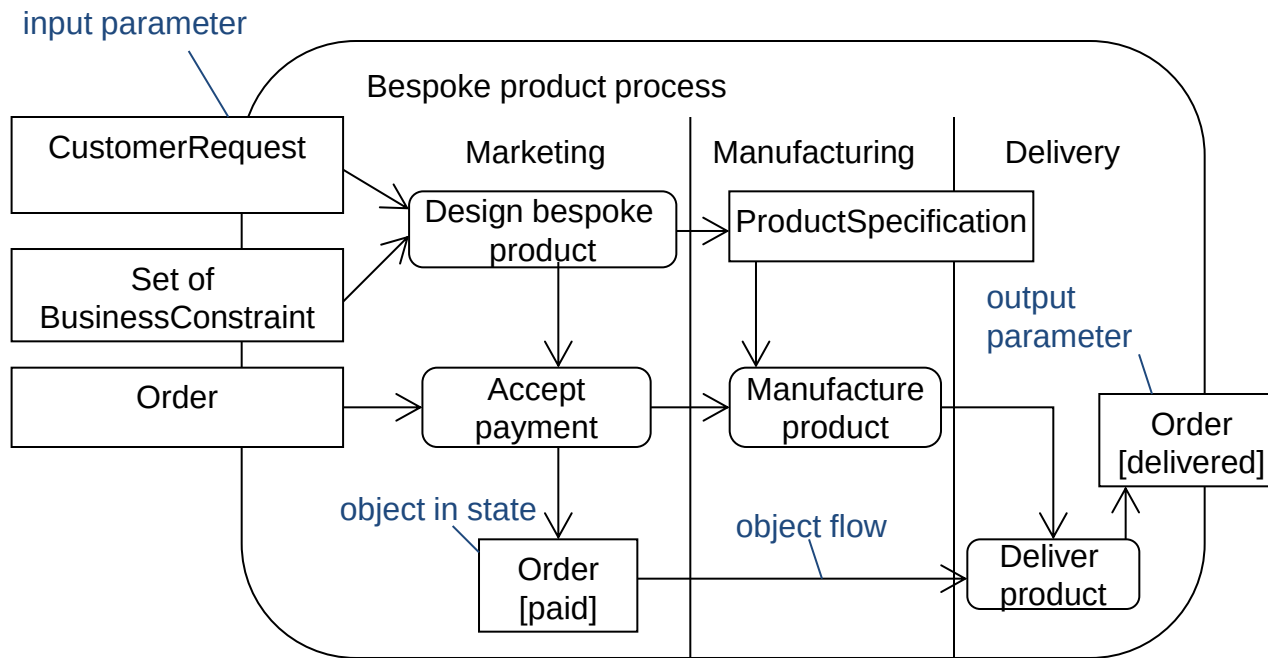
Order

Order objects raised in December may be available

Order
[open]

select Order objects in the open state

Activity parameters and partitioning

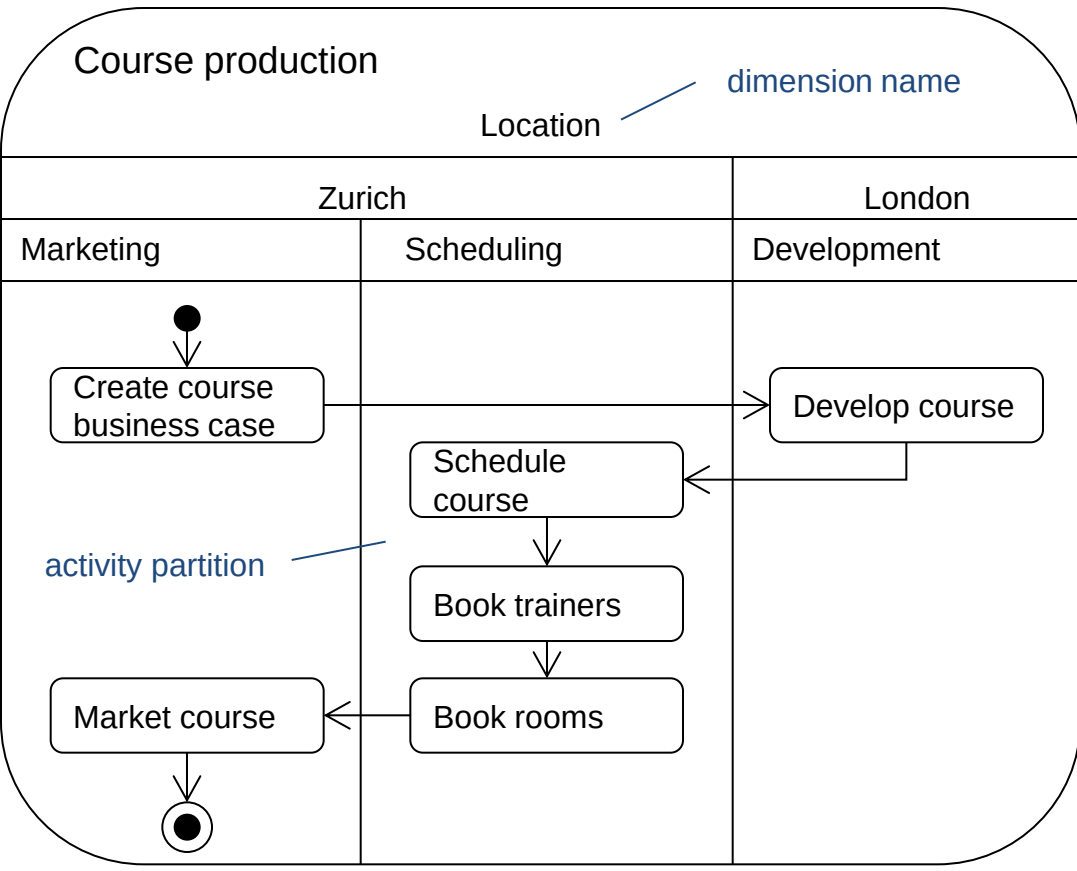


- ✧ Object nodes can provide input and output parameters to activities
 - Input parameters have one or more output object flows into the activity
 - Output parameters have one or more input object flows out of the activity
- ✧ Draw the object node overlapping the activity boundary

Activity partitions



- Each activity partition represents a high-level grouping of a set of related actions
 - Partitions can be hierarchical
 - Partitions can be vertical, horizontal or both
- Partitions can refer to many different things e.g. business organisations, classes, components and so on
- If partitions can't be shown clearly using parallel lines, put their name in brackets directly above the name of the activities



(London::Marketing)
Market product

(p1, p2)
SomeAction

nested partitions

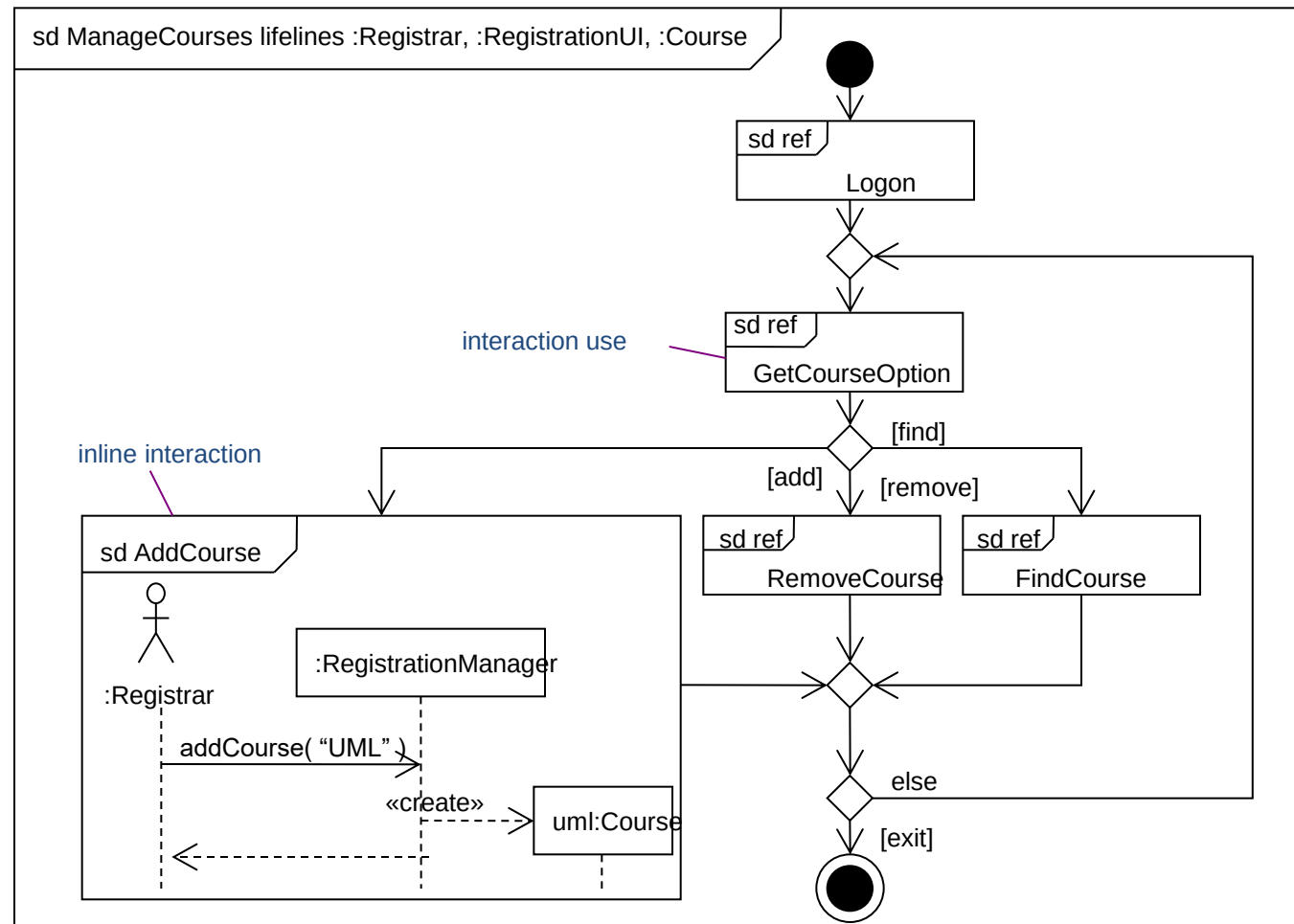
multiple partitions



Interaction overview diagrams



- ✧ Model the high level flow of control between interactions
- ✧ Show interactions and interaction occurrences
- ✧ Have activity diagram syntax



Key points



✧ Activity diagrams can model flows of activities using:

- Activities and connectors
 - Call action node
 - Send signal/accept event action node
 - Accept time event action node
- Control nodes
 - Decision and merge
 - Fork and join
- Object nodes
 - Input and output parameters
 - Pins

✧ Interaction overview diagrams as their advanced feature